

CSC311: Data Structures

Stacks

Dr. Amlan Chatterjee

Computer Science Department
California State University, Dominguez Hills

Stack

- Objectives
- To learn about the stack data type and how to use its four methods:
 - push
 - pop
 - peek
 - empty
- To learn how to implement a stack using an underlying array
- To see how to use a stack to perform various applications, including finding palindromes, testing for balanced (properly nested) parentheses, and evaluating arithmetic expressions

Stack Abstract Data Type

- A stack is one of the most commonly used data structures in computer science
- A stack can be compared to a Pez dispenser
 - Only the top item can be accessed
 - You can extract only one item at a time
- The top element in the stack is the last added to the stack (most recently)
- The stack's storage policy is *Last-In, First-Out*, or *LIFO*



Specification of the Stack Abstract Data Type

- Only the top element of a stack is visible; therefore the number of operations performed by a stack are few
- We need the ability to
 - test for an empty stack (`empty`)
 - inspect the top element (`peek`)
 - retrieve the top element (`pop`)
 - put a new element on the stack (`push`)

Methods	Behavior
<code>boolean empty()</code>	Returns true if the stack is empty; otherwise, returns false .
<code>E peek()</code>	Returns the object at the top of the stack without removing it.
<code>E pop()</code>	Returns the object at the top of the stack and removes it.
<code>E push(E obj)</code>	Pushes an item onto the top of the stack and returns the item pushed.

A Stack of Strings

Jonathan
Dustin
Robin
Debbie
Rich

(a)

Dustin
Robin
Debbie
Rich

(b)

Philip
Dustin
Robin
Debbie
Rich

(c)

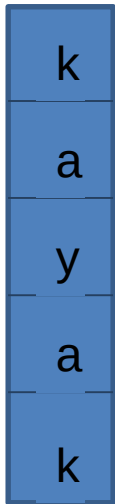
- “Rich” is the oldest element on the stack and “Jonathan” is the youngest (Figure a)
- `String last = names.peek();` stores a reference to “Jonathan” in `last`
- `String temp = names.pop();` removes “Jonathan” and stores a reference to it in `temp` (Figure b)
- `names.push("Philip");` pushes “Philip” onto the stack (Figure c)

Stack Applications: Finding Palindromes

- Palindrome: a string that reads identically in either direction, letter by letter (ignoring case etc.)
 - “kayak”
 - “racecar”
 - “Never odd or even”
 - “Level madam level”
- Problem: Write a program that reads a string and determines whether it is a palindrome

Finding Palindromes (cont.)

- Solving using a stack:
 - Push each string character, from left to right, onto a stack



k a y a k

```
private void fillStack() {  
    for(int i = 0; i < inputString.length(); i++) {  
        charStack.push(inputString.charAt(i));  
    }  
}
```

Finding Palindromes (cont.)

- Solving using a stack:
 - Pop each character off the stack, appending each to the `StringBuilder result`



k a y a k

```
private String buildReverse(){
    StringBuilder result = new StringBuilder();
    while(!charStack.empty()) {
        result.append(charStack.pop());
    }
    return result.toString();
}
```

Testing

- To test this class using the following inputs:
 - a single character (always a palindrome)
 - multiple characters in a word
 - multiple words
 - different cases
 - even-length strings
 - odd-length strings
 - the empty string (considered a palindrome)

Balanced Parentheses

- When analyzing arithmetic expressions, it is important to determine whether an expression is balanced with respect to parentheses

$$(a + b * (c / (d - e))) + (d / e)$$

- The problem is further complicated if braces or brackets are used in conjunction with parentheses
- The solution is to use stacks!

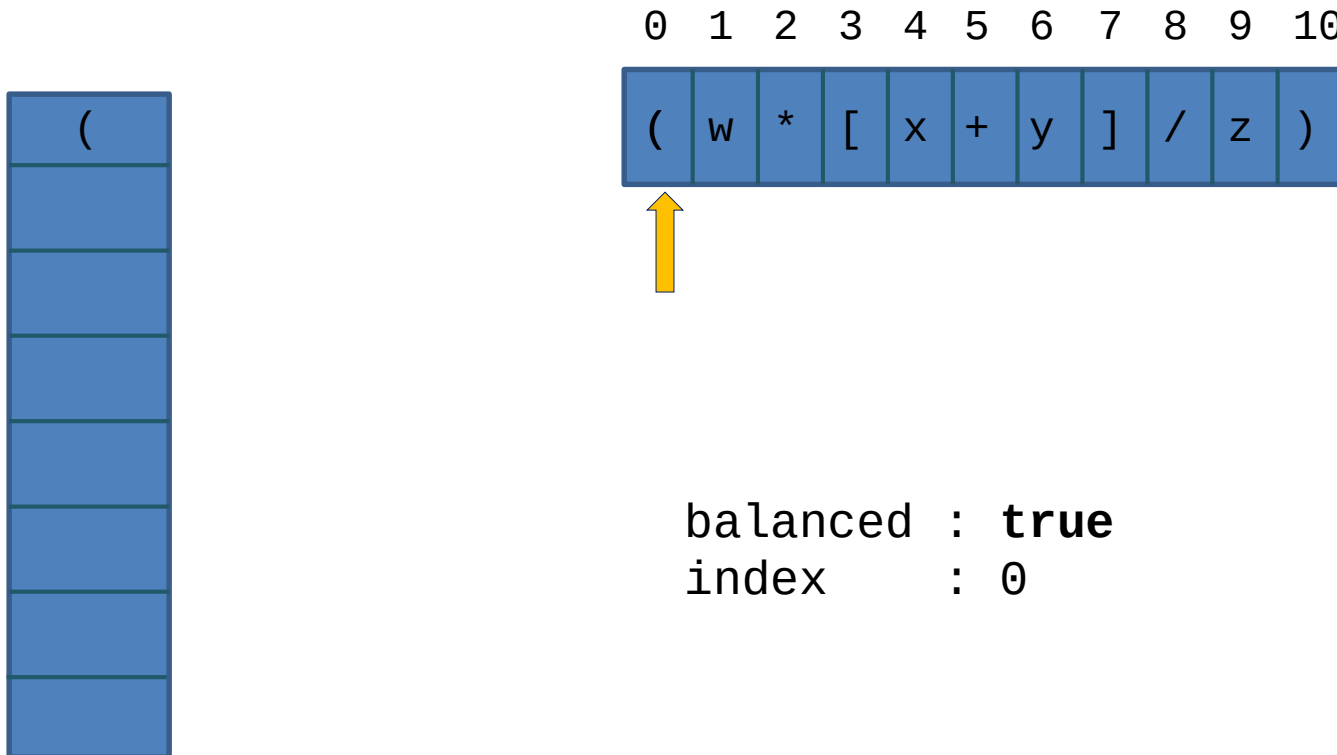
Balanced Parentheses (cont.)

Algorithm for method `isBalanced`

1. Create an empty stack of characters.
2. Assume that the expression is balanced (`balanced` is `true`).
3. Set `index` to 0.
4. **while** `balanced` is `true` and `index` < the expression's length
5. Get the next character in the data string.
6. **if** the next character is an opening parenthesis
7. Push it onto the stack.
8. **else if** the next character is a closing parenthesis
9. Pop the top of the stack.
10. **if** stack was empty or its top does not match the closing parenthesis
11. Set `balanced` to `false`.
12. Increment `index`.
13. Return `true` if `balanced` is `true` and the stack is empty.

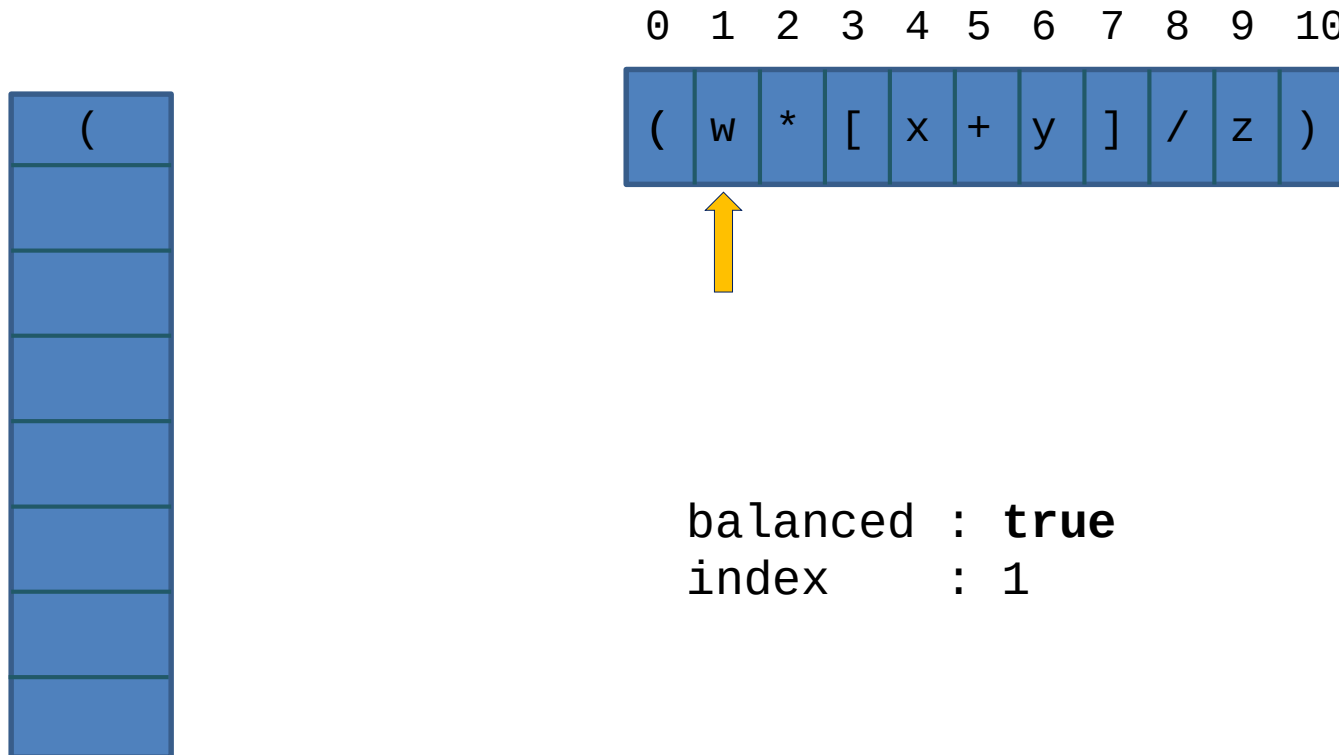
Balanced Parentheses (cont.)

Expression: $(w * [x + y] / z)$



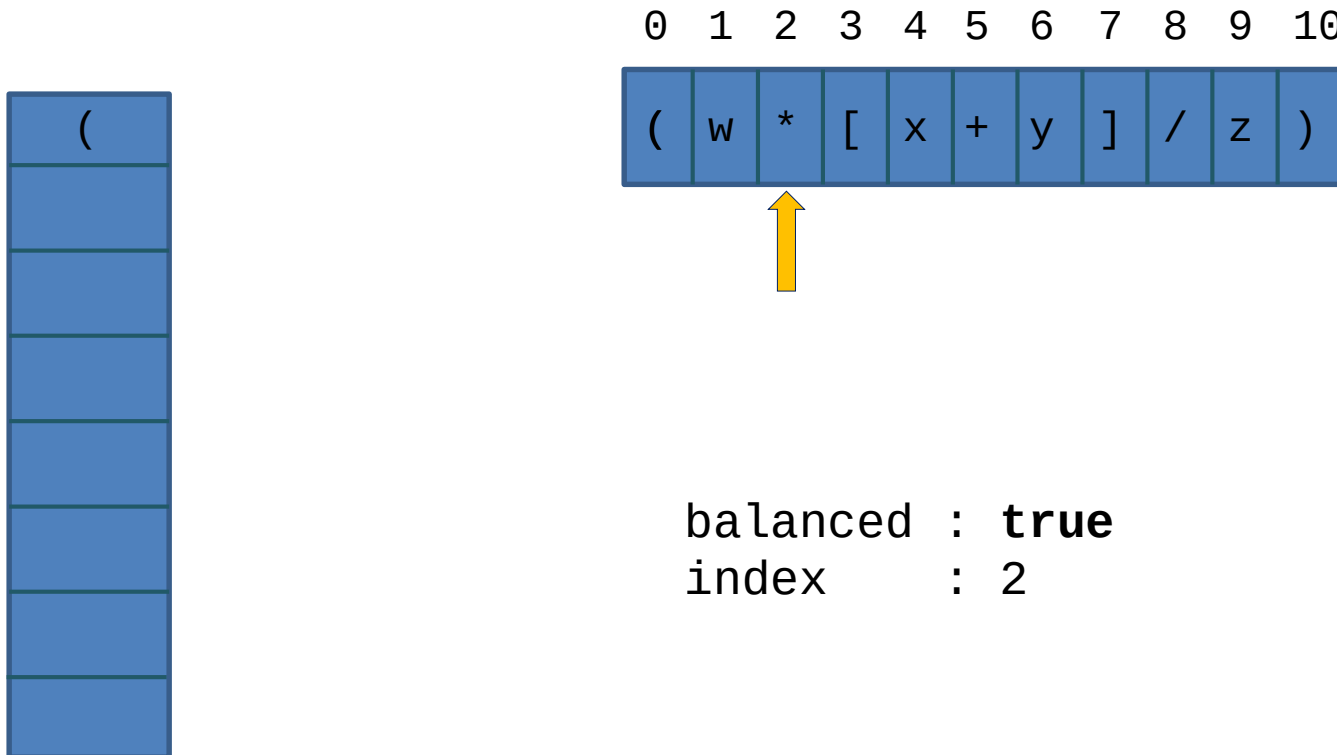
Balanced Parentheses (cont.)

Expression: $(w * [x + y] / z)$



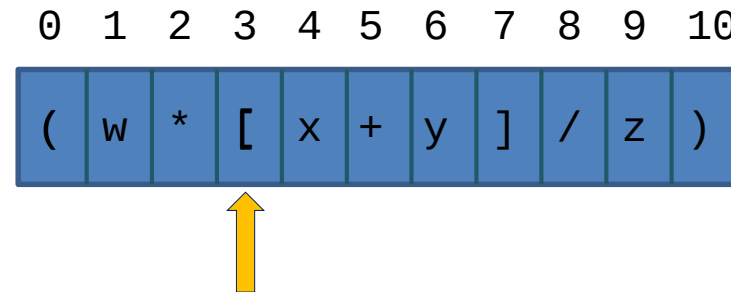
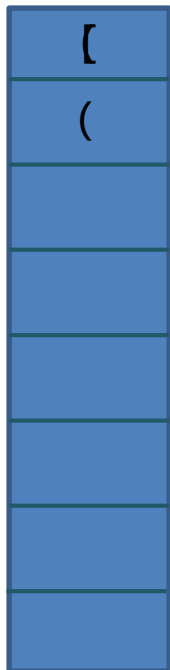
Balanced Parentheses (cont.)

Expression: (w * [x + y] / z)



Balanced Parentheses (cont.)

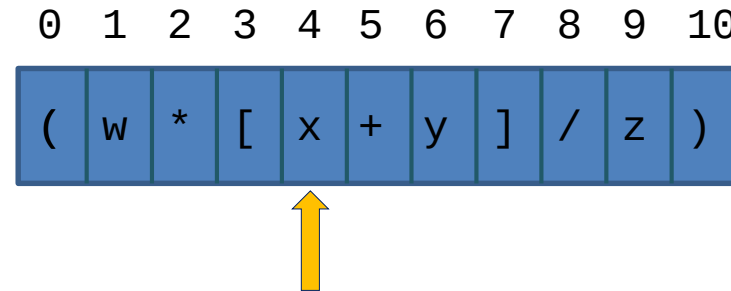
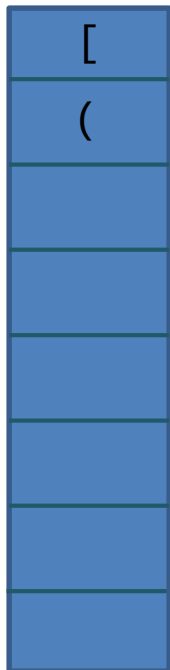
Expression: (w * [x + y] / z)



balanced : **true**
index : 3

Balanced Parentheses (cont.)

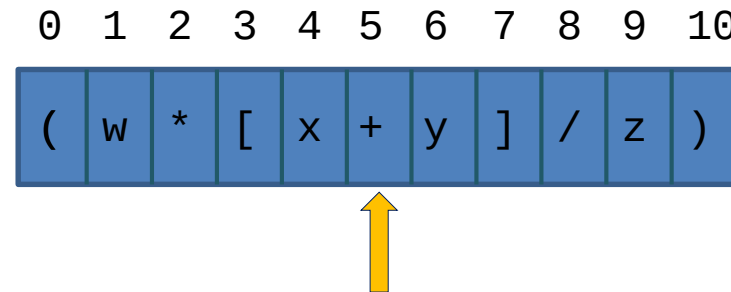
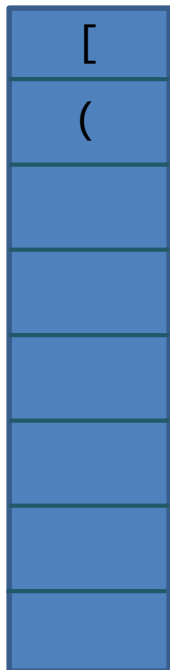
Expression: (w * [x + y] / z)



balanced : **true**
index : 4

Balanced Parentheses (cont.)

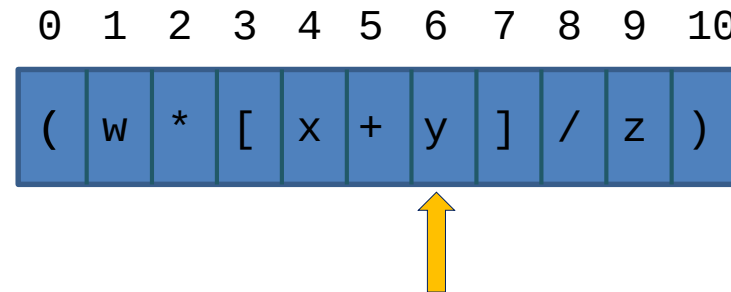
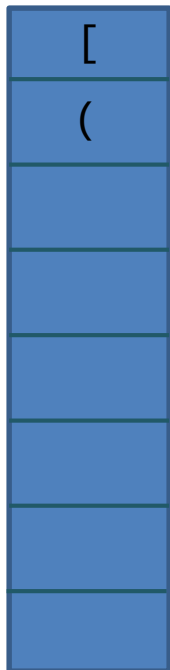
Expression: $(w * [x + y] / z)$



balanced : **true**
index : 5

Balanced Parentheses (cont.)

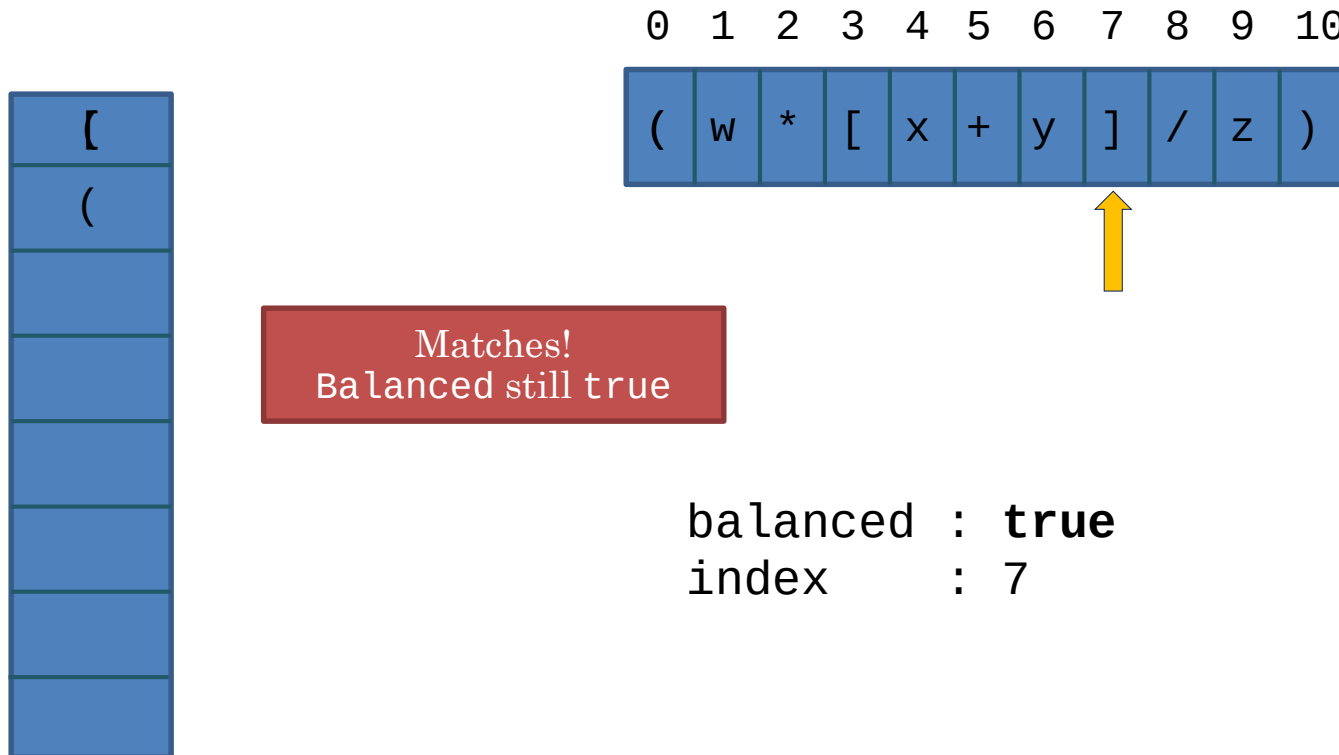
Expression: (w * [x + y] / z)



balanced : **true**
index : 6

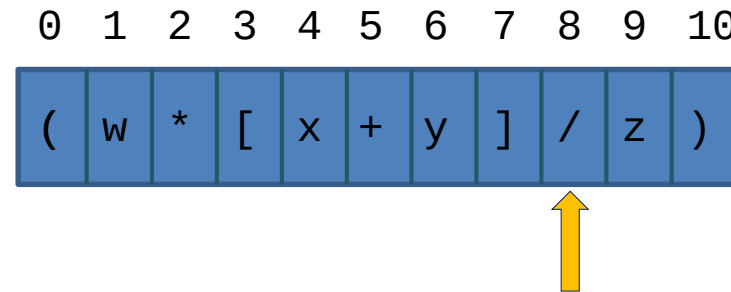
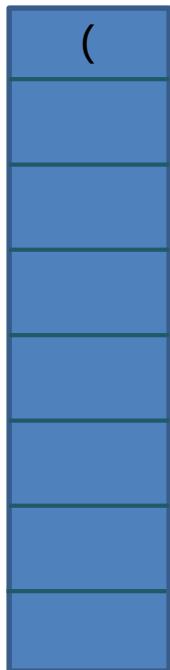
Balanced Parentheses (cont.)

Expression: (w * [x + y] / z)



Balanced Parentheses (cont.)

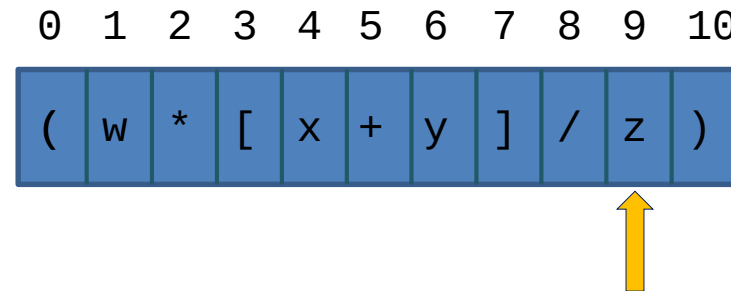
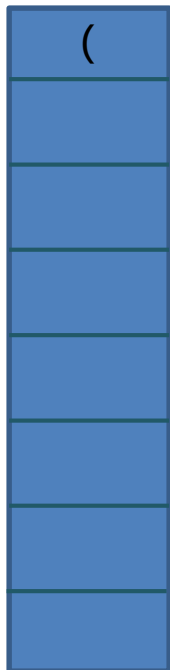
Expression: (w * [x + y] / z)



balanced : **true**
index : 8

Balanced Parentheses (cont.)

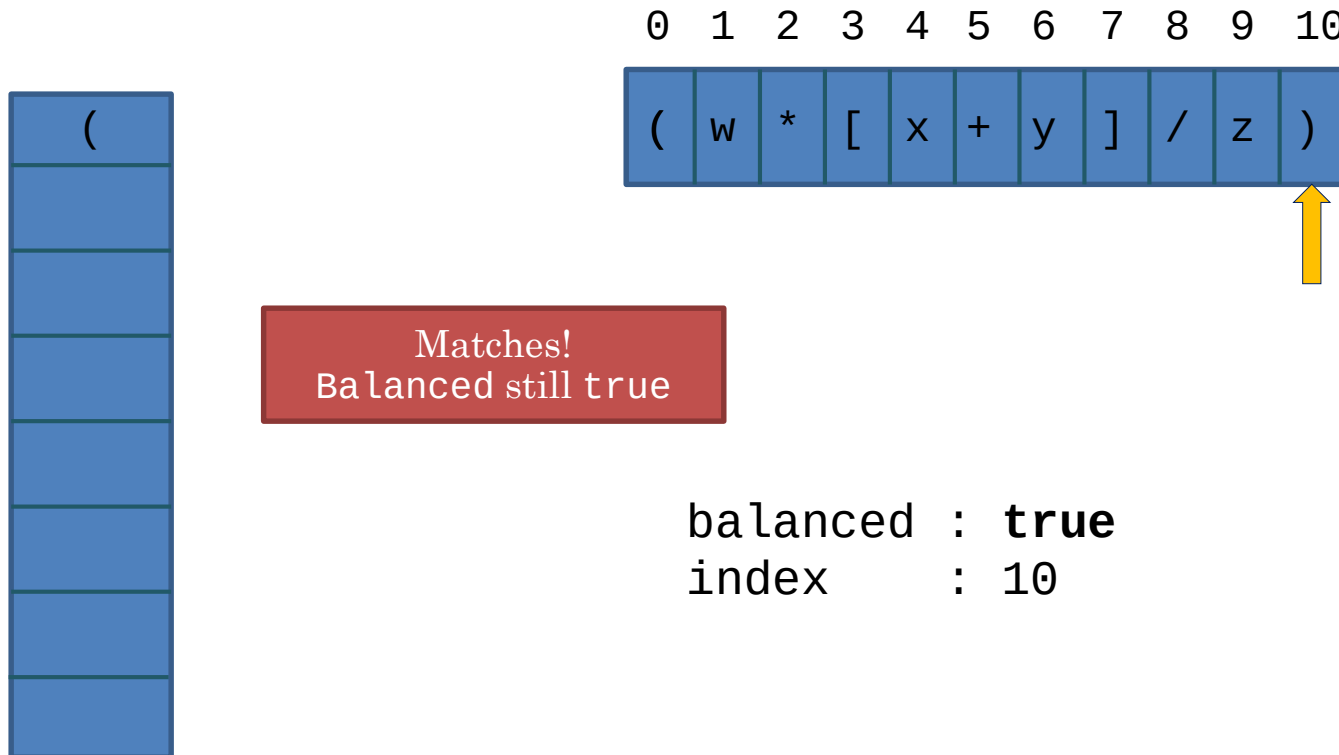
Expression: (w * [x + y] / z)



balanced : **true**
index : 9

Balanced Parentheses (cont.)

Expression: (w * [x + y] / z)



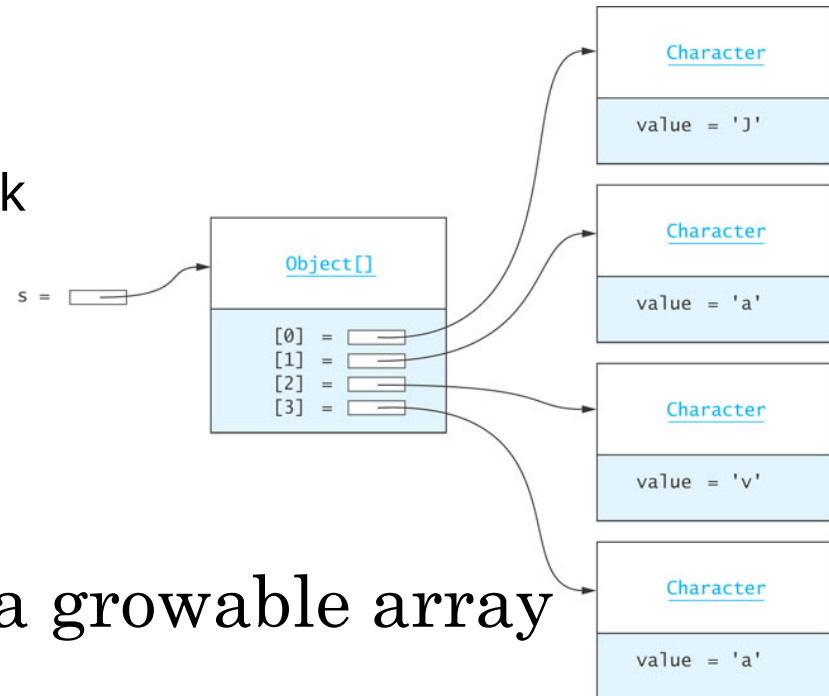
Testing

- Provide a variety of input expressions displaying the result `true` or `false`
- Try several levels of nested parentheses
- Try nested parentheses where corresponding parentheses are not of the same type
- Try unbalanced parentheses
- No parentheses at all!

Implementing a Stack as an Extension of Vector

- The Java API includes a stack class as part of the package `java.util` :

```
public class Stack<E> extends Vector<E>
```
- The vector class implements a growable array of objects
- Elements of a vector can be accessed using an integer index and the size can grow or shrink as needed to accommodate the insertion and removal of elements



Implementing a Stack as an Extension of Vector

- We can use vector's add method to implement push:

```
public E push(obj E) {  
    add(obj);  
    return obj;  
}
```

- pop can be coded as

```
public E pop throws EmptyStackException {  
    try {  
        return remove (size() - 1);  
    } catch (ArrayIndexOutOfBoundsException ex) {  
        throw new EmptyStackException();  
    }  
}
```

Implementing a Stack as an Extension of Vector

- Because a Stack *is a* vector, all of vector operations can be applied to a stack (such as searches and access by index)
- But, since only the top element of a stack should be accessible, this violates the principle of information hiding

Implementing a Stack Using an Array

- If we implement a stack as an array, we would need . . .

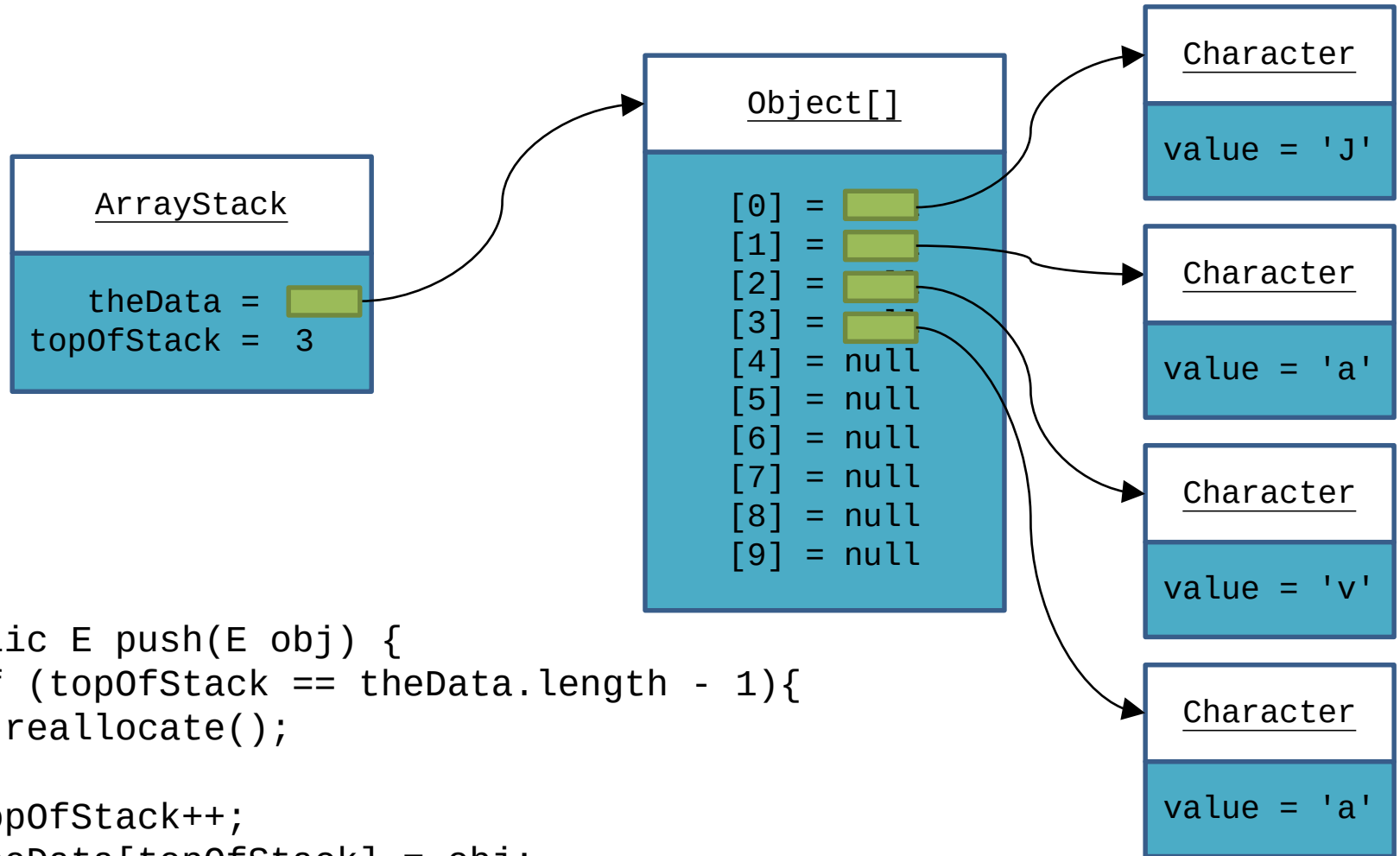
```
public class ArrayStack<E> implements Stack<E> {
    private E[] theData;
    int topOfStack = -1;
    private static final int INITIAL_CAPACITY = 10;

    @SuppressWarnings("unchecked")
    public ArrayStack() {
        theData = (E[])new Object[INITIAL_CAPACITY];
    }
}
```

Allocate storage for an array with a default

Keep track of the top of the stack (subscript of the element at the top of the stack; for empty stack = -1)

Implementing a Stack Using an Array



```
public E push(E obj) {  
    if (topOfStack == theData.length - 1){  
        reallocate();  
    }  
    topOfStack++;  
    theData[topOfStack] = obj;  
    return obj;  
}
```

Implementing a Stack Using an Array

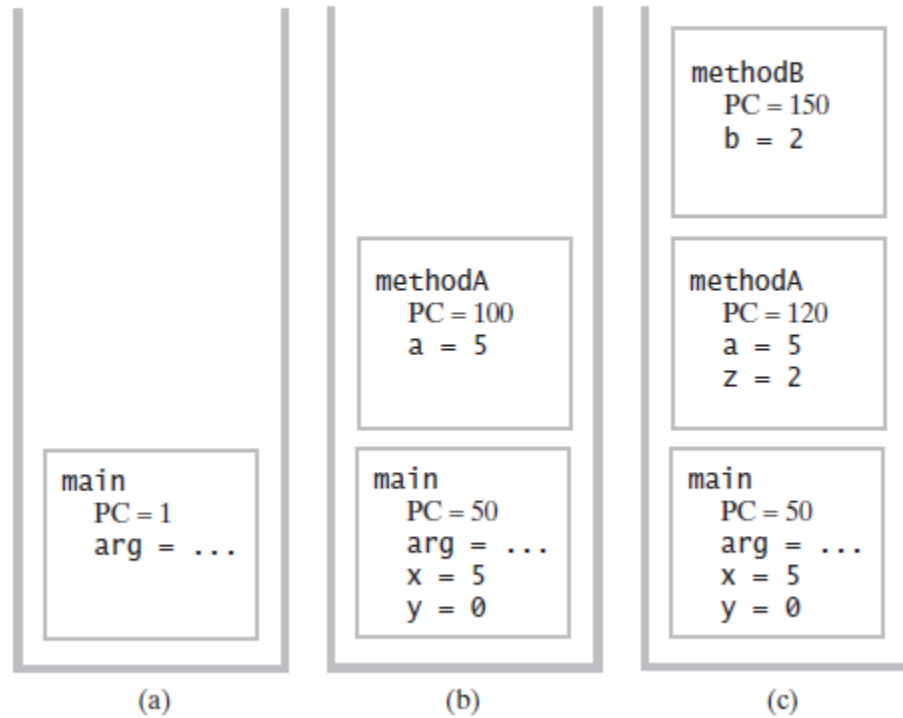
```
@Override
public E pop() {
    if (empty()) {
        throw new EmptyStackException();
    }
    return theData[topOfStack--];
}
```

The Program Stack

```
1  public static
   void main(string[] arg)
   {
       int x = 5;
50  int y = methodA(x);
   } // end main

100 public static
    int methodA(int a)
    {
       int z = 2;
120  methodB(z);
       return z;
   } // end methodA

150 public static
    void methodB(int b)
    {
   } // end methodB
```



Program

Program stack at three points in time (PC is the program counter)

The program stack at three points in time: (a) when main begins execution; (b) when `methodA` begins execution; (c) when `methodB` begins execution

Interview Question

- How would you design a stack which, in addition to push and pop, also has a function min which returns the minimum element? Push, pop and min should all operate in $O(1)$ time.

Processing Algebraic Expressions

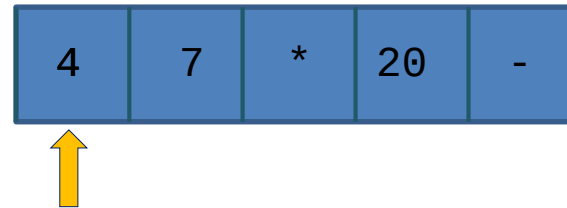
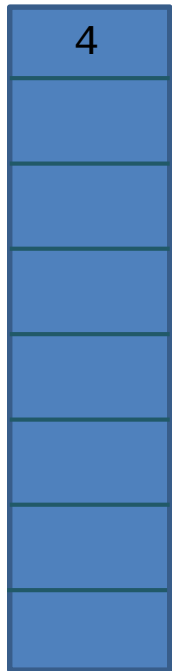
- Infix: each binary operator appears between its operands $a + b$
- Prefix: each binary operator appears before its operands $+ a b$
- Postfix: each binary operator appears after its operands $a b +$

Additional Stack Applications

- Postfix and infix notation
 - Expressions normally are written in infix form, but
 - it easier to evaluate an expression in postfix form since there is no need to group sub-expressions in parentheses or worry about operator precedence

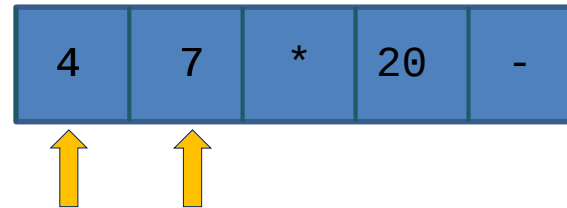
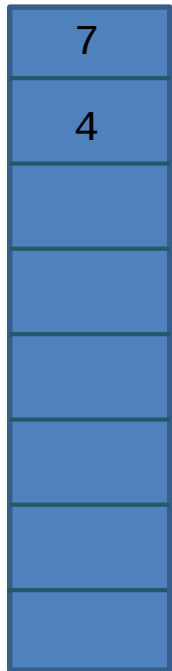
Postfix Expression	Infix Expression	Value
$\underline{4 \quad 7 \quad *}$	$4 * 7$	28
$\underline{4 \quad \underline{7 \quad 2 \quad +} \quad *}$	$4 * (7 + 2)$	36
$\underline{\underline{4 \quad 7 \quad *}} \quad 20 \quad -$	$(4 * 7) - 20$	8
$\underline{3 \quad \underline{\underline{4 \quad 7 \quad *}} \quad 2 \quad /} \quad +$	$3 + ((4 * 7) / 2)$	17

Evaluating Postfix Expressions



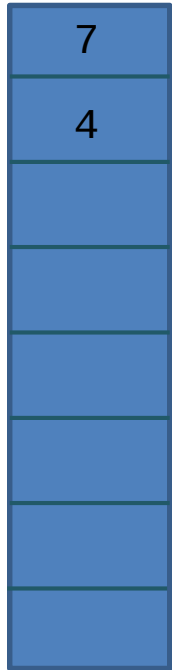
- ➔ 1. create an empty stack of integers
- ➔ 2. while there are more tokens
- ➔ 3. get the next token
- ➔ 4. if the first character of the token is a digit
- ➔ 5. push the token on the stack
- 6. else if the token is an operator
- 7. pop the right operand off the stack
- 8. pop the left operand off the stack
- 9. evaluate the operation
- 10. push the result onto the stack
- 11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)

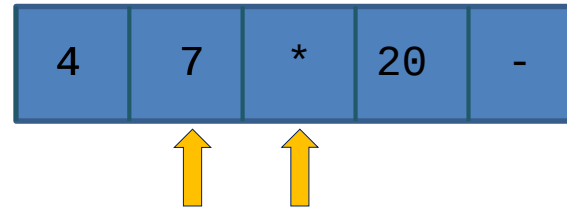


1. create an empty stack of integers
- 2. while there are more tokens
3. get the next token
- 4. if the first character of the token is a digit
- 5. push the token on the stack
6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
9. evaluate the operation
10. push the result onto the stack
11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)

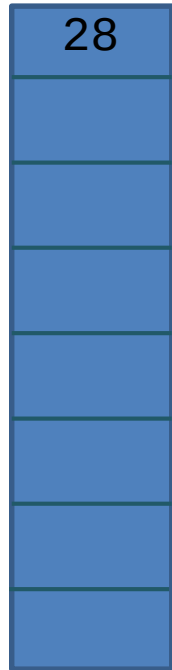


4 * 7

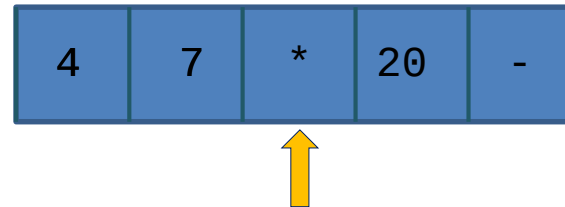


1. create an empty stack of integers
- 2. while there are more tokens
3. get the next token
- 4. if the first character of the token is a digit
5. push the token on the stack
- 6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
- 9. evaluate the operation
10. push the result onto the stack
11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)

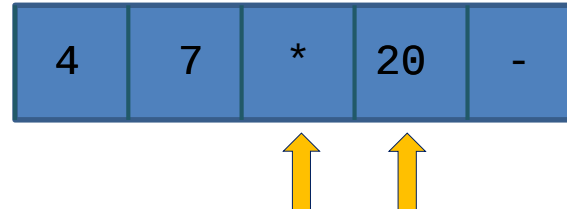
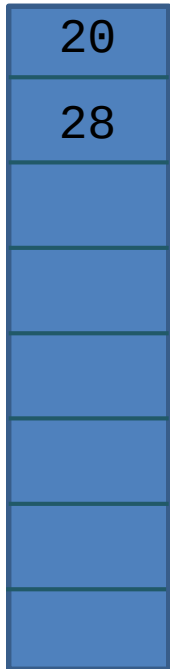


28



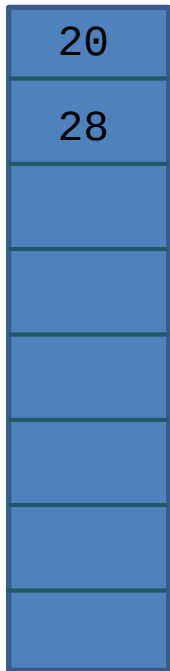
1. create an empty stack of integers
2. while there are more tokens
3. get the next token
4. if the first character of the token is a digit
5. push the token on the stack
6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
- 9. evaluate the operation
- 10. push the result onto the stack
11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)

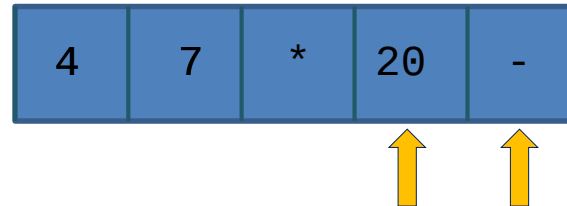


1. create an empty stack of integers
- 2. while there are more tokens
3. get the next token
- 4. if the first character of the token is a digit
- 5. push the token on the stack
6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
9. evaluate the operation
10. push the result onto the stack
11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)

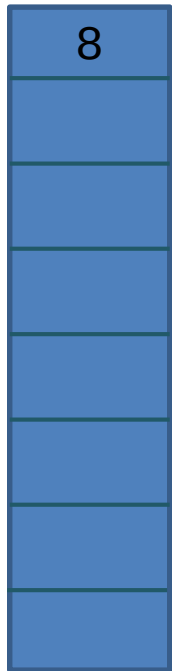


28 - 20



1. create an empty stack of integers
- 2. while there are more tokens
3. get the next token
- 4. if the first character of the token is a digit
5. push the token on the stack
- 6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
- 9. evaluate the operation
10. push the result onto the stack
11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)

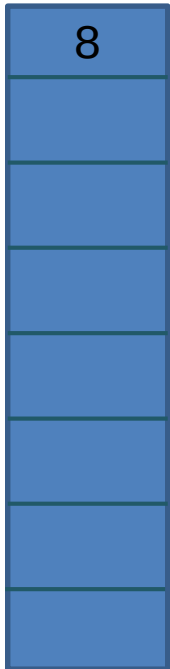
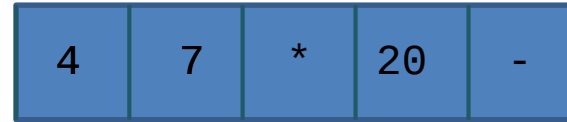


8



1. create an empty stack of integers
2. while there are more tokens
3. get the next token
4. if the first character of the token is a digit
5. push the token on the stack
6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
- 9. evaluate the operation
- 10. push the result onto the stack
11. pop the stack and return the result

Evaluating Postfix Expressions (cont.)



1. create an empty stack of integers
- 2. while there are more tokens
3. get the next token
4. if the first character of the token is a digit
5. push the token on the stack
6. else if the token is an operator
7. pop the right operand off the stack
8. pop the left operand off the stack
9. evaluate the operation
10. push the result onto the stack
- 11. pop the stack and return the result

Converting from Infix to Postfix

- Convert infix expressions to postfix expressions
- Assume:
 - expressions consists of only spaces, operands, and operators
 - space is a delimiter character
 - all operands that are identifiers begin with a letter or underscore
 - all operands that are numbers begin with a digit

Converting from Infix to Postfix (cont.)

- Example: convert
$$w - 5.1 / \text{sum} * 2$$
to its postfix form
$$w\ 5.1\ \text{sum} / 2 * -$$

Converting from Infix to Postfix (cont.)

Algorithm for Method convert

1. Initialize postfix to an empty StringBuilder.
2. Initialize the operator stack to an empty stack.
3. **while** there are more tokens in the infix string
4. Get the next token.
5. **if** the next token is an operand
6. Append it to postfix.
7. **else if** the next token is an operator
8. Call processOperator to process the operator.
9. **else**
10. Indicate a syntax error.
11. Pop remaining operators off the operator stack and append them to postfix.

Converting from Infix to Postfix (cont.)

Algorithm for Method processOperator

1. **if** the operator stack is empty
2. Push the current operator onto the stack.
- else**
3. Peek the operator stack and let topOp be the top operator.
4. **if** the precedence of the current operator is greater than the
 precedence of topOp
5. Push the current operator onto the stack.
- else**
6. **while** the stack is not empty and the precedence of the current
 operator is less than or equal to the precedence of topOp
7. Pop topOp off the stack and append it to postfix.
8. **if** the operator stack is not empty
9. Peek the operator stack and let topOp be the top
 operator.
10. Push the current operator onto the stack.

Converting from Infix to Postfix (cont.)

Next Token	Action	Effect on operatorStack	Effect on postfix
w	Append w to postfix.		w
-	The stack is empty Push - onto the stack		w
5.1	Append 5.1 to postfix		w 5.1
/	precedence(/) > precedence(-), Push / onto the stack		w 5.1
sum	Append sum to postfix		w 5.1 sum
*	precedence(*) equals precedence(/) Pop / off of stack and append to postfix		w 5.1 sum /

Converting from Infix to Postfix (cont.)

Next Token	Action	Effect on operatorStack	Effect on postfix
*	precedence(*) > precedence(-), Push * onto the stack	* -	w 5.1 sum /
2	Append 2 to postfix	* -	w 5.1 sum / 2
End of input	Stack is not empty, Pop * off the stack and append to postfix	-	w 5.1 sum / 2 *
End of input	Stack is not empty, Pop - off the stack and append to postfix		w 5.1 sum / 2 * -

- Questions..??

Interview Question

- Given an integer array, find the number which is not repeated; all others are present for two times.
- Ex: `int [ ] A = {2, -9, 100, -9, 21, 5, 2, 5, 100}`
- Answer: 21